

Word Break (Medium)

Given a string *s* and a dictionary of strings *wordDict*, return true if *s* can be segmented into a space-separated sequence of one or more dictionary words.

Constraints:

- $1 \leq s.length \leq 300$
- $1 \leq wordDict.length \leq 1000$
- $1 \leq wordDict[i].length \leq 20$
- *s* and *wordDict[i]* consist of only lowercase English letters.
- All the strings of *wordDict* are **unique**.

Approach:

To tackle this difficulty, we can use dynamic programming. We'll iterate through the given string *s*, checking to see if any of its prefixes can be divided into words from the dictionary. We'll keep track of the segmentation options for various *s* prefixes using a dynamic programming array. Specifically, if a *s* prefix can be divided into words from the dictionary, we will mark that place in the DP array as True.

```
Code
Java Auto
1 class Solution {
2     public boolean wordBreak(String s, List<String> wordDict) {
3         return wordBreak(s, new HashSet<>(wordDict), new HashMap<>());
4     }
5
6     private boolean wordBreak(final String s, Set<String> wordSet, Map<String, Boolean> memo) {
7         if (memo.containsKey(s))
8             return memo.get(s);
9         if (wordSet.contains(s)) {
10             memo.put(s, true);
11             return true;
12         }
13
14         // 1 <= prefix.length() < s.length()
15         for (int i = 1; i < s.length(); ++i) {
16             final String prefix = s.substring(0, i);
17             final String suffix = s.substring(i);
18             if (wordSet.contains(prefix) && wordBreak(suffix, wordSet, memo)) {
19                 memo.put(s, true);
20                 return true;
21             }
22         }
23
24         memo.put(s, false);
25         return false;
26     }
27 }
```

Saved Ln 24, Col 24

Explanation:

1. We create a **dp array**, where $dp[i]$ tells us if the first i characters of the string can be split into valid words from the dictionary.
2. As a starting point, we set $dp[0] = \text{True}$ because an empty string is always considered valid.
3. To speed things up, we put all the words from the dictionary into a **set** for quick lookups.
4. Then, we go through the string character by character. For each position i , we check if the substring up to that point ($s[:i]$) can be broken down into valid words.
5. To figure this out, we try splitting $s[:i]$ at every possible spot j . If the left part ($s[:j]$) is valid ($dp[j] = \text{True}$) **and** the right part ($s[j:i]$) is in the dictionary, then we know $s[:i]$ is valid, so we mark $dp[i] = \text{True}$ and stop checking further.
6. After going through the entire string, the answer will be in $dp[n]$, where n is the length of the string. If it's True , the whole string can be segmented; otherwise, it can't.

The screenshot shows the LeetCode interface for the 'Word Break' problem (139). The problem description states: Given a string s and a dictionary of strings $wordDict$, return true if s can be segmented into a space-separated sequence of one or more dictionary words. A note mentions that the same word in the dictionary may be reused multiple times. Examples include 'leetcode' being segmented as 'leet code' and 'applepenapple' as 'apple pen apple'. Constraints specify limits on string length, dictionary size, and word length. A Java solution is provided, using a recursive approach with memoization to check if a string can be segmented. The test results show three cases, all of which are 'Accepted' with a runtime of 0 ms.

139. Word Break

Medium Topics Companies

Given a string s and a dictionary of strings $wordDict$, return true if s can be segmented into a space-separated sequence of one or more dictionary words.

Note that the same word in the dictionary may be reused multiple times in the segmentation.

Example 1:

Input: $s = \text{"leetcode"}, wordDict = [\text{"leet"}, \text{"code"}]$
Output: true
Explanation: Return true because "leetcode" can be segmented as "leet code".

Example 2:

Input: $s = \text{"applepenapple"}, wordDict = [\text{"apple"}, \text{"pen"}]$
Output: true
Explanation: Return true because "applepenapple" can be segmented as "apple pen apple". Note that you are allowed to reuse a dictionary word.

Example 3:

Input: $s = \text{"catsandog"}, wordDict = [\text{"cats"}, \text{"dog"}, \text{"sand"}, \text{"and"}, \text{"cat"}]$
Output: false

Constraints:

- $1 \leq s.length \leq 300$
- $1 \leq wordDict.length \leq 1000$
- $1 \leq wordDict[i].length \leq 20$

18.3K 251 166 Online

```
class Solution {
    public boolean wordBreak(String s, List<String> wordDict) {
        return wordBreak(s, new HashSet<>(wordDict), new HashMap<>());
    }

    private boolean wordBreak(final String s, Set<String> wordSet, Map<String, Boolean> memo) {
        if (memo.containsKey(s)) {
            return memo.get(s);
        }
        if (wordSet.contains(s)) {
            memo.put(s, true);
            return true;
        }
    }
}
```

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

$s = \text{"catsandog"}$

$wordDict = [\text{"cats"}, \text{"dog"}, \text{"sand"}, \text{"and"}, \text{"cat"}]$

Output

false

Expected

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

$s = \text{"leetcode"}$

$wordDict = [\text{"leet"}, \text{"code"}]$

Output

true

Expected

true

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

$s = \text{"applepenapple"}$

$wordDict = [\text{"apple"}, \text{"pen"}]$

Output

true

Expected

true

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

$s = \text{"catsandog"}$

$wordDict = [\text{"cats"}, \text{"dog"}, \text{"sand"}, \text{"and"}, \text{"cat"}]$

Output

false

Expected

false